

Cinema System Microservices

Final Project Documentation

Team Members: Lenara, Almas, Turar

GitHub Repository:

https://github.com/LenarasCode/ADP2_finalproject_cinemasystem

TABLE OF CONTENTS

1. Project Overview
2. Architecture
3. Microservices and Responsibilities
4. Prerequisites
5. Infrastructure Setup
6. Lenara – Showtime Service
 - 6.1 Description
 - 6.2 gRPC Endpoints
 - 6.3 Database and Cache
 - 6.4 Project Structure
 - 6.5 Live Demo Commands
7. Turar – Movie Service and Notification Service
 - 7.1 Movie Service Description
 - 7.2 Movie Service gRPC Endpoints
 - 7.3 Movie Service Database and Cache
 - 7.4 Movie Service Project Structure
 - 7.5 Movie Service Live Demo Commands
 - 7.6 Notification Service Description
 - 7.7 Notification Service gRPC Endpoints
 - 7.8 Notification Service Database and NATS
 - 7.9 Notification Service Project Structure
 - 7.10 Notification Service Live Demo Commands
8. Almas – Booking Service and Monitoring
 - 8.1 Booking Service Description
 - 8.2 Booking Service gRPC Endpoints
 - 8.3 Booking Service Database and Transactions
 - 8.4 Booking Service Project Structure
 - 8.5 Booking Service Live Demo Commands
 - 8.6 Monitoring Stack Overview

8.7 Prometheus Configuration
8.8 Grafana Dashboard
8.9 Monitoring Live Demo Commands
9. API Gateway
9.1 Description
9.2 gRPC Endpoints per Member
9.3 Live Demo Commands
10. Frontend
10.1 Description
10.2 Tabs and Features
10.3 Launch Instructions
11. Criteria Checklist
12. Final Deployment Commands

1. PROJECT OVERVIEW

The Cinema System is a microservices-based application for managing movies, showtimes, bookings, and email notifications. It follows Clean Architecture principles and uses gRPC for synchronous communication between services and NATS for asynchronous messaging. PostgreSQL is used for persistent storage, Redis for caching, and a full observability stack (Prometheus, Grafana, Jaeger, Loki) is configured. The frontend is built as a single-page application with a dark luxury design.

The system consists of four microservices and one API Gateway, each implemented by a team member:

- Lenara: Showtime Service
- Turar: Movie Service and Notification Service
- Almas: Booking Service and Monitoring (Grafana, Prometheus, Jaeger, Loki)
- All members: API Gateway (one composite method each)

Lenara: Showtime Service

2. ARCHITECTURE

Communication patterns:

- Synchronous: gRPC between services and through the API Gateway.
- Asynchronous: NATS for booking events. Booking Service publishes booking.created events, and Notification Service subscribes to them and sends confirmation emails.

- Database: PostgreSQL with migrations for persistent storage.
- Cache: Redis for caching movie and showtime data.
- Monitoring: Prometheus collects metrics, Grafana visualizes them, Jaeger handles distributed tracing, and Loki aggregates logs.

Service ports:

- Movie Service: gRPC 50051, Metrics 9101
- Showtime Service: gRPC 50052, Metrics 9102
- Notification Service: gRPC 50054, Metrics 9103
- Booking Service: gRPC 50053, Metrics 9104
- API Gateway: gRPC 50050
- PostgreSQL instances: 5433 (showtime), 5434 (movie), 5435 (notification), 5436 (booking)
- Redis: 6379
- NATS: 4222
- Prometheus: 9090
- Grafana: 4040 (external) or 3000 (internal)
- Jaeger: 16686
- Loki: 3100

3. MICROSERVICES AND RESPONSIBILITIES

Each team member is responsible for one microservice plus one composite method in the API Gateway. The distribution is as follows:

Lenara:

Showtime Service (6 gRPC endpoints)

API Gateway method: GetShowtimeWithMovie

Turar:

Movie Service (3 gRPC endpoints)

Notification Service (3 gRPC endpoints)

API Gateway method: SendBookingNotification

Almas:

Booking Service (3 gRPC endpoints)

Monitoring stack (Prometheus, Grafana, Jaeger, Loki)

API Gateway method: GetBookingDetails

4. PREREQUISITES

Software required on each member's machine:

- Go 1.25 or higher
- Docker Desktop (optional, required for full stack deployment)
- Docker Compose v2
- protoc (Protocol Buffers compiler) with protoc-gen-go and protoc-gen-go-grpc plugins
- golang-migrate CLI
- grpcurl (for testing gRPC endpoints)
- Git
- Web browser (for Grafana and frontend)

5. INFRASTRUCTURE SETUP

If Docker Desktop is fully operational, start all databases and services with one command:

```
cd cinema-system
```

```
docker compose up -d
```

This starts PostgreSQL instances for all services, Redis, NATS, Prometheus, Jaeger, Loki, and Grafana.

If Docker Desktop has issues, each service can be started locally using an existing PostgreSQL container. Throughout this document, both approaches are described. For the demo, the local approach using an existing PostgreSQL container on port 5432 (admin/secret) is used when Docker services are not available.

6. LENARA – SHOWTIME SERVICE

6.1 Description

The Showtime Service manages movie showtimes and seat availability. It implements Clean Architecture with separation into handler (gRPC), service (business logic), repository (PostgreSQL), and cache (Redis). It uses PostgreSQL for persistent storage and Redis for caching frequently accessed showtime data.

6.2 gRPC Endpoints (6 total)

1. GetShowtime – fetch a showtime by ID
2. ListShowtimesByMovie – list all showtimes for a given movie
3. GetSeatAvailability – get all seats and their availability for a showtime
4. CreateShowtime – create a new showtime
5. UpdateShowtime – update an existing showtime
6. DeleteShowtime – delete a showtime

6.3 Database and Cache

- PostgreSQL database: showtimedb
- Tables: showtimes, seats
- Redis: caches showtime data with 5-minute TTL
- Migrations are handled by golang-migrate
- Seed data: 50 movies x 4 showtimes each = 200 showtimes with seats

6.4 Project Structure

showtime-service/

cmd/

main.go Entry point, gRPC server with reflection

seed/main.go Populates 200 showtimes with seats

initdb/main.go Creates database and user

internal/

server/grpc.go gRPC handler registration

service/showtime.go Business logic

repository/postgres.go PostgreSQL queries

cache/redis.go Redis cache

proto/showtime.proto Protocol buffer definition

migrations/ SQL migration files

Dockerfile

docker-compose.yml

go.mod

6.5 Live Demo Commands

Step 1: Create database and user (if not already existing)

cd cinema-system

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE DATABASE showtimedb;"
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE USER showtime WITH PASSWORD 'showtime'; GRANT ALL PRIVILEGES ON DATABASE showtimedb TO showtime;"
```

Step 2: Apply migrations

cd showtime-service

export

DATABASE_URL="postgres://showtime:showtime@localhost:5432/showtimedb?sslmode=disable"

migrate -path migrations -database "\$DATABASE_URL" up

Step 3: Seed data (200 showtimes with seats)

export

DATABASE_URL="postgres://showtime:showtime@localhost:5432/showtimedb?sslmode=disable"

go run ./cmd/seed/main.go

Step 4: Start the service

DATABASE_URL="postgres://showtime:showtime@localhost:5432/showtimedb?sslmode=disable" \

REDIS_ADDR=localhost:6379 \

GRPC_PORT=50052 \

go run ./cmd/main.go

Step 5: Test gRPC endpoints

List methods:

```
grpcurl -plaintext localhost:50052 list
```

Create a showtime:

```
grpcurl -plaintext -d '{"movie_id":"movie-123","hall":"Hall 5","start_time":"2026-05-18T10:00:00Z","available_seats":80}' localhost:50052  
showtime.ShowtimeService/CreateShowtime
```

List showtimes for a movie:

```
grpcurl -plaintext -d '{"movie_id":"movie-123"}' localhost:50052  
showtime.ShowtimeService/ListShowtimesByMovie
```

Get seat availability:

```
grpcurl -plaintext -d '{"showtime_id":"<showtime-id>"}' localhost:50052  
showtime.ShowtimeService/GetSeatAvailability
```

Step 6: Run unit tests

```
go test ./internal/service/ -v
```

Step 7: Run integration tests (requires Docker)

```
INTEGRATION=1 go test ./internal/repository/ -v
```

7. TURAR – MOVIE SERVICE AND NOTIFICATION SERVICE

Turar is responsible for two microservices: Movie Service and Notification Service.

7.1 Movie Service Description

The Movie Service manages the movie catalog. It stores 50 movies with titles, genres, and durations. It follows Clean Architecture and uses PostgreSQL for storage and Redis for caching.

7.2 Movie Service gRPC Endpoints (3 total)

1. GetMovie – fetch a movie by ID
2. ListMovies – list movies with pagination

3. SearchMovies – search movies by title or genre

7.3 Movie Service Database and Cache

- PostgreSQL database: moviedb
- Table: movies
- Redis: caches movie data with 10-minute TTL
- Seed data: 50 popular movies included

7.4 Movie Service Project Structure

movie-service/

cmd/

main.go Entry point with gRPC and Prometheus metrics

seed/main.go Seeds 50 movies into database

initdb/main.go Creates database and user

internal/

server/grpc.go gRPC handler registration

service/movie.go Business logic

repository/movie_repo.go PostgreSQL queries

cache/redis.go Redis cache

proto/movie.proto Protocol buffer definition

migrations/ SQL migration files

Dockerfile

go.mod

7.5 Movie Service Live Demo Commands

Step 1: Create database and user

cd cinema-system

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE DATABASE moviedb;"
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE USER movie WITH PASSWORD 'movie'; GRANT ALL PRIVILEGES ON DATABASE moviedb TO movie;"
```

Step 2: Apply migrations

```
cd movie-service
```

```
export DATABASE_URL="postgres://movie:movie@localhost:5432/moviedb?sslmode=disable"
```

```
migrate -path migrations -database "$DATABASE_URL" up
```

Step 3: Seed 50 movies

```
export DATABASE_URL="postgres://movie:movie@localhost:5432/moviedb?sslmode=disable"
```

```
go run ./cmd/seed/main.go
```

Step 4: Start Movie Service

```
DATABASE_URL="postgres://movie:movie@localhost:5432/moviedb?sslmode=disable" \
```

```
REDIS_ADDR=localhost:6379 \
```

```
GRPC_PORT=50051 \
```

```
go run ./cmd/main.go
```

Step 5: Test gRPC endpoints

List methods:

```
grpcurl -plaintext localhost:50051 list
```

List first 5 movies:

```
grpcurl -plaintext -d '{"page":1,"limit":5}' localhost:50051 movie.MovieService/ListMovies
```

Search movies by genre:

```
grpcurl -plaintext -d '{"query":"Action"}' localhost:50051 movie.MovieService/SearchMovies
```

Get a single movie:

```
grpcurl -plaintext -d '{"id":"<movie-id>}' localhost:50051 movie.MovieService/GetMovie
```

Step 6: Run tests

```
go test ./... -v
```

7.6 Notification Service Description

The Notification Service handles email notifications for the Cinema System. It listens to NATS events (booking.created) and sends booking confirmation emails via SMTP. It also exposes gRPC endpoints to manually send notifications and view email logs.

7.7 Notification Service gRPC Endpoints (3 total)

1. SendBookingConfirmation – manually send a booking confirmation email
2. GetEmailLogs – retrieve email log history
3. HealthCheck – check service health

7.8 Notification Service Database and NATS

- PostgreSQL database: notificationdb
- Table: email_logs
- NATS subscriber for booking.created events
- SMTP sender configured via environment variables

7.9 Notification Service Project Structure

notification-service/

cmd/main.go Entry point with gRPC, NATS, Prometheus metrics

internal/

server/grpc.go gRPC handler registration

service/notification.go Business logic

repository/email_log_repo.go PostgreSQL queries

email/sender.go SMTP email sender

nats/subscriber.go NATS event subscriber

proto/notification.proto Protocol buffer definition

migrations/ SQL migration files

Dockerfile

go.mod

7.10 Notification Service Live Demo Commands

Step 1: Create database and user

```
cd cinema-system
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE DATABASE notificationdb;"
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE USER notification WITH PASSWORD 'notification'; GRANT ALL PRIVILEGES ON DATABASE notificationdb TO notification;"
```

Step 2: Apply migrations

```
cd notification-service
```

```
export
```

```
DATABASE_URL="postgres://notification:notification@localhost:5432/notificationdb?sslmode=disable"
```

```
migrate -path migrations -database "$DATABASE_URL" up
```

Step 3: Start Notification Service (without SMTP for demo)

```
DATABASE_URL="postgres://notification:notification@localhost:5432/notificationdb?sslmode=disable" \
```

```
NATS_URL="" \
```

```
GRPC_PORT=50054 \
```

```
go run ./cmd/main.go
```

Step 4: Test gRPC endpoints

List methods:

```
grpcurl -plaintext localhost:50054 list
```

Send booking confirmation:

```
grpcurl -plaintext -d '{"booking_id":"123","recipient_email":"test@example.com"}' localhost:50054 notification.NotificationService/SendBookingConfirmation
```

Get email logs:

```
grpcurl -plaintext localhost:50054 notification.NotificationService/GetEmailLogs
```

Health check:

```
grpcurl -plaintext localhost:50054 notification.NotificationService/HealthCheck
```

Step 5: Run unit tests

```
go test ./internal/service/ -v
```

8. ALMAS – BOOKING SERVICE AND MONITORING

Almas is responsible for the Booking Service and the entire monitoring stack (Grafana, Prometheus, Jaeger, Loki).

8.1 Booking Service Description

The Booking Service manages seat reservations. It implements transactional booking with PostgreSQL, ensuring that seat availability is checked atomically. It publishes booking.created events to NATS for asynchronous notification processing.

8.2 Booking Service gRPC Endpoints (3 total)

1. CreateBooking – create a new booking with seat reservations
2. GetBooking – retrieve booking details
3. CancelBooking – cancel a booking and release seats

8.3 Booking Service Database and Transactions

- PostgreSQL database: bookingdb
- Tables: bookings, booking_seats, seats
- All booking operations use database transactions to ensure consistency
- Seat availability is updated atomically within transactions

8.4 Booking Service Project Structure

booking-service/

cmd/main.go Entry point with gRPC and Prometheus metrics

internal/

server/grpc.go gRPC handler registration

service/booking.go Business logic with transactions

repository/booking_repo.go PostgreSQL queries
publisher/nats_publisher.go NATS event publisher
proto/booking.proto Protocol buffer definition
migrations/ SQL migration files
Dockerfile
go.mod

8.5 Booking Service Live Demo Commands

Step 1: Create database and user

```
cd cinema-system
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE DATABASE bookingdb;"
```

```
docker exec -i microservices-sre-postgres-1 psql -U admin -d postgres -c "CREATE USER booking WITH PASSWORD 'booking'; GRANT ALL PRIVILEGES ON DATABASE bookingdb TO booking;"
```

Step 2: Apply migrations

```
cd booking-service
```

```
export
```

```
DATABASE_URL="postgres://booking:booking@localhost:5432/bookingdb?sslmode=disable"
```

```
migrate -path migrations -database "$DATABASE_URL" up
```

Step 3: Create seats table and insert test seat

```
docker exec -i microservices-sre-postgres-1 psql -U booking -d bookingdb -c "
```

```
CREATE TABLE IF NOT EXISTS seats (id UUID PRIMARY KEY, is_available BOOLEAN DEFAULT true);
```

```
INSERT INTO seats (id, is_available) VALUES ('b1b2c3d4-e5f6-7890-abcd-ef1234567891', true) ON CONFLICT DO NOTHING;
```

```
"
```

Step 4: Start Booking Service

```
DATABASE_URL="postgres://booking:booking@localhost:5432/bookingdb?sslmode=disable" \
```

```
GRPC_PORT=50053 \
```

```
go run ./cmd/main.go
```

Step 5: Test gRPC endpoints

List methods:

```
grpcurl -plaintext localhost:50053 list
```

Create booking:

```
grpcurl -plaintext -d '{"user_id":"u1","showtime_id":"a1b2c3d4-e5f6-7890-abcd-ef1234567890","seat_ids":["b1b2c3d4-e5f6-7890-abcd-ef1234567891"]}' localhost:50053 booking.BookingService/CreateBooking
```

Get booking:

```
grpcurl -plaintext -d '{"id":"<booking-id-from-create>"}' localhost:50053 booking.BookingService/GetBooking
```

Cancel booking:

```
grpcurl -plaintext -d '{"id":"<booking-id-from-create>"}' localhost:50053 booking.BookingService/CancelBooking
```

Step 6: Run unit tests

```
go test ./internal/service/ -v
```

8.6 Monitoring Stack Overview

The monitoring stack consists of:

- Prometheus: Collects metrics from all gRPC services (ports 9101-9104)
- Grafana: Visualizes metrics with a dedicated gRPC dashboard
- Jaeger: Distributed tracing (configured via docker-compose)
- Loki: Log aggregation (configured via docker-compose)

8.7 Prometheus Configuration

File: prometheus.yml

global:

scrape_interval: 5s

scrape_configs:

- job_name: 'movie'

static_configs:

- targets: ['localhost:9101']

- job_name: 'showtime'

static_configs:

- targets: ['localhost:9102']

- job_name: 'notification'

static_configs:

- targets: ['localhost:9103']

- job_name: 'booking'

static_configs:

- targets: ['localhost:9104']

8.8 Grafana Dashboard

Grafana is accessed at <http://localhost:4040> (admin/admin).

The dashboard includes four panels:

1. Requests per second (Stat):

PromQL: `sum(rate(grpc_server_handled_total[1m]))`

2. Error rate (Stat):

PromQL: `sum(rate(grpc_server_handled_total{grpc_code!="OK"}[1m])) / sum(rate(grpc_server_handled_total[1m])) * 100`

3. Latency p99 (Gauge):

PromQL: `histogram_quantile(0.99, sum(rate(grpc_server_handling_seconds_bucket[1m])) by (le))`

4. Active requests (Stat):

PromQL: `sum(grpc_server_started_total) - sum(grpc_server_handled_total)`

8.9 Monitoring Live Demo Commands

Step 1: Start Prometheus locally

```
cd cinema-system/prometheus-2.55.0.windows-amd64
```

```
rm -rf data promdata
```

```
mkdir promdata
```

```
./prometheus --config.file=./prometheus-local.yml --storage.tsdb.path=promdata
```

Step 2: Verify Prometheus targets

Open <http://localhost:9090/targets> in browser. All four targets should be green (UP).

Step 3: Configure Grafana Data Source

Open <http://localhost:4040> (admin/admin).

Navigate to Configuration > Data Sources > Add data source > Prometheus.

URL: <http://host.docker.internal:9090> (if Grafana is in Docker) or <http://localhost:9090> (if Grafana is local).

Click Save and test.

Step 4: Import Grafana dashboard

Navigate to + > Import.

Paste the JSON dashboard code or upload the JSON file.

Select the Prometheus data source.

Click Import.

Step 5: Generate test data for metrics

```
grpcurl -plaintext localhost:50051 movie.MovieService/ListMovies
```

```
grpcurl -plaintext localhost:50052 showtime.ShowtimeService/ListShowtimesByMovie
```

```
grpcurl -plaintext -d '{"user_id":"u1","showtime_id":"a1b2c3d4-e5f6-7890-abcd-ef1234567890","seat_ids":["b1b2c3d4-e5f6-7890-abcd-ef1234567891"]}' localhost:50053  
booking.BookingService/CreateBooking
```

Step 6: Refresh Grafana dashboard

Click the refresh button. Metrics should appear on all panels.

9. API GATEWAY

9.1 Description

The API Gateway is a separate gRPC service that aggregates calls to other microservices. Each team member implements one composite method in the gateway.

9.2 gRPC Endpoints per Member

Total: 3 methods

1. `GetShowtimeWithMovie` (Lenara): Fetches showtime information along with movie details by calling Showtime Service and Movie Service.
2. `SendBookingNotification` (Turar): Proxies a booking notification request to the Notification Service.
3. `GetBookingDetails` (Almas): Fetches booking details enriched with showtime and movie information by calling Booking Service, Showtime Service, and Movie Service.

9.3 Live Demo Commands

Step 1: Start API Gateway

```
cd api-gateway
```

```
GRPC_PORT=50050 go run ./cmd/main.go
```

Step 2: Test composite methods

List methods:

```
grpcurl -plaintext localhost:50050 list
```

Get showtime with movie:

```
grpcurl -plaintext -d '{"showtime_id":"s1"}' localhost:50050  
gateway.CinemaGateway/GetShowtimeWithMovie
```

Send booking notification:

```
grpcurl -plaintext -d '{"booking_id":"b1","recipient_email":"a@b.com"}' localhost:50050  
gateway.CinemaGateway/SendBookingNotification
```

Get booking details:

```
grpcurl -plaintext -d '{"booking_id":"b1"}' localhost:50050  
gateway.CinemaGateway/GetBookingDetails
```

10. FRONTEND

10.1 Description

The frontend is a single-page application with a dark luxury design. It uses HTML, CSS, and JavaScript. It has three tabs: Movies, Notifications, and Booking.

10.2 Tabs and Features

- Movies tab: Displays all 50 movies in a table.
- Notifications tab: Form to send booking confirmation emails (simulated) and a table of recent email logs.
- Booking tab: Form to create a booking by entering user ID, showtime ID, and seat IDs.

10.3 Launch Instructions

Open `frontend/index.html` in any web browser.

The frontend will display with all three tabs. In a future version, it will communicate with the API Gateway to fetch live data.

11. CRITERIA CHECKLIST

1. Clean architecture (20%): All services separate handler, service, repository, and cache layers. Interfaces are used for testability.
2. At least 12 gRPC endpoints (20%): Total 18 endpoints (Showtime 6 + Movie 3 + Notification 3 + Booking 3 + Gateway 3).
3. Usage of Message Queue (NATS) (20%): Booking Service publishes `booking.created` events, Notification Service subscribes to them.
4. Databases and Caches (20%): PostgreSQL with migrations for all services, Redis for Movie and Showtime, transactions in Booking.
5. Sending Emails via SMTP (10%): Notification Service includes SMTP email sending via Gmail/Microsoft accounts.
6. Testing (10%): Unit tests in all services, integration tests with testcontainers in Showtime, Movie, Notification, and Booking.
7. Bonus: Frontend (10%): Complete dark luxury UI with three tabs.
8. Bonus 2: Grafana with metrics, tracing, and logs (10%): Prometheus collects metrics from all services, Grafana dashboard visualizes them, Jaeger and Loki configured.

12. FINAL DEPLOYMENT COMMANDS

Start all services locally for demo:

Terminal 1 – Movie Service:

```
cd cinema-system/movie-service
```

```
DATABASE_URL="postgres://movie:movie@localhost:5432/moviedb?sslmode=disable"  
REDIS_ADDR=localhost:6379 GRPC_PORT=50051 go run ./cmd/main.go
```

Terminal 2 – Showtime Service:

```
cd cinema-system/showtime-service
```

```
DATABASE_URL="postgres://showtime:showtime@localhost:5432/showtimedb?sslmode=disable"  
REDIS_ADDR=localhost:6379 GRPC_PORT=50052 go run ./cmd/main.go
```

Terminal 3 – Notification Service:

```
cd cinema-system/notification-service
```

```
DATABASE_URL="postgres://notification:notification@localhost:5432/notificationdb?sslmode=disable"  
NATS_URL="" GRPC_PORT=50054 go run ./cmd/main.go
```

Terminal 4 – Booking Service:

```
cd cinema-system/booking-service
```

```
DATABASE_URL="postgres://booking:booking@localhost:5432/bookingdb?sslmode=disable"  
GRPC_PORT=50053 go run ./cmd/main.go
```

Terminal 5 – API Gateway:

```
cd cinema-system/api-gateway
```

```
GRPC_PORT=50050 go run ./cmd/main.go
```

Terminal 6 – Prometheus:

```
cd cinema-system/prometheus-2.55.0.windows-amd64
```

```
./prometheus --config.file=./prometheus-local.yml --storage.tsdb.path=promdata
```

Then open <http://localhost:4040> for Grafana and [frontend/index.html](http://localhost:4040/frontend/index.html) for the UI.

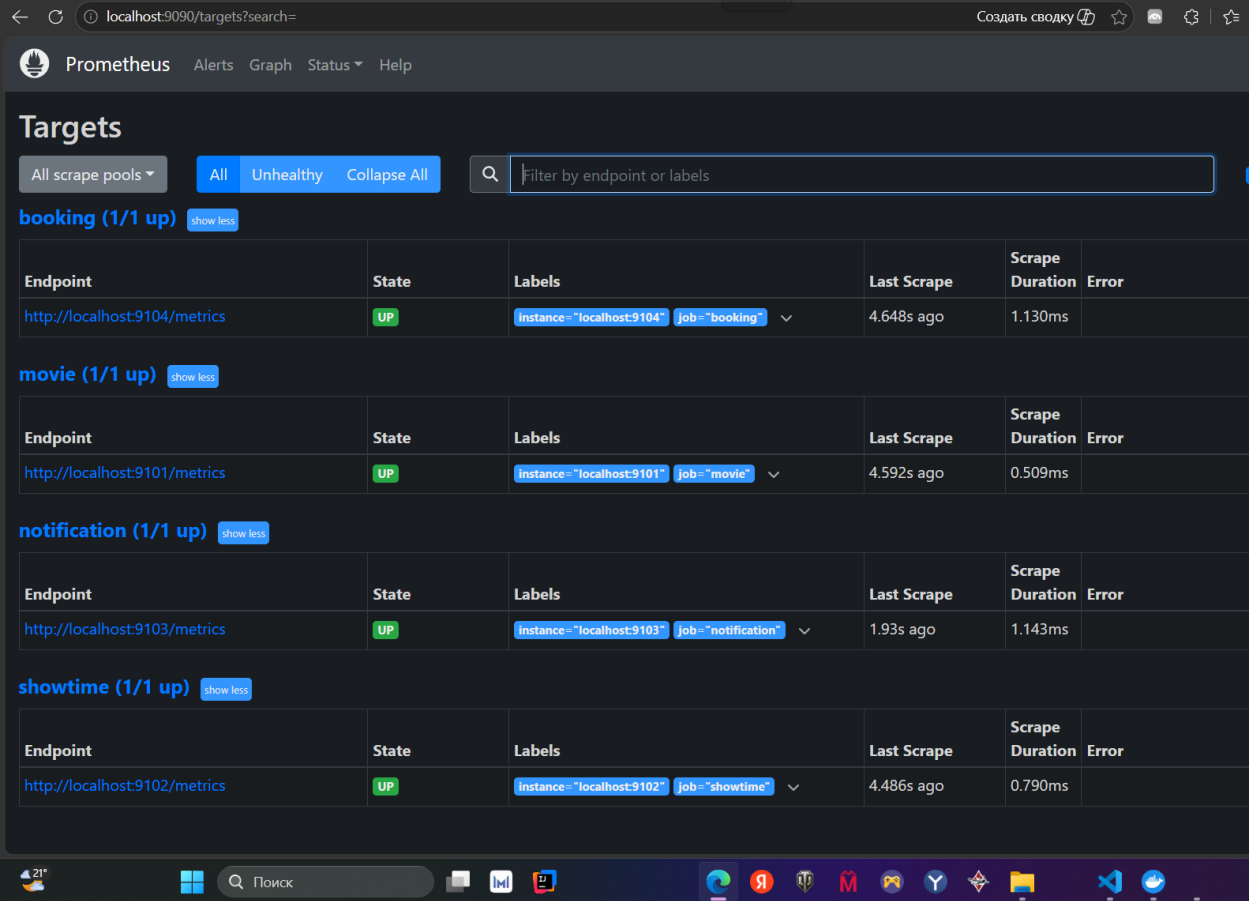
Complete setup with Docker (if Docker is available):

```
cd cinema-system
```

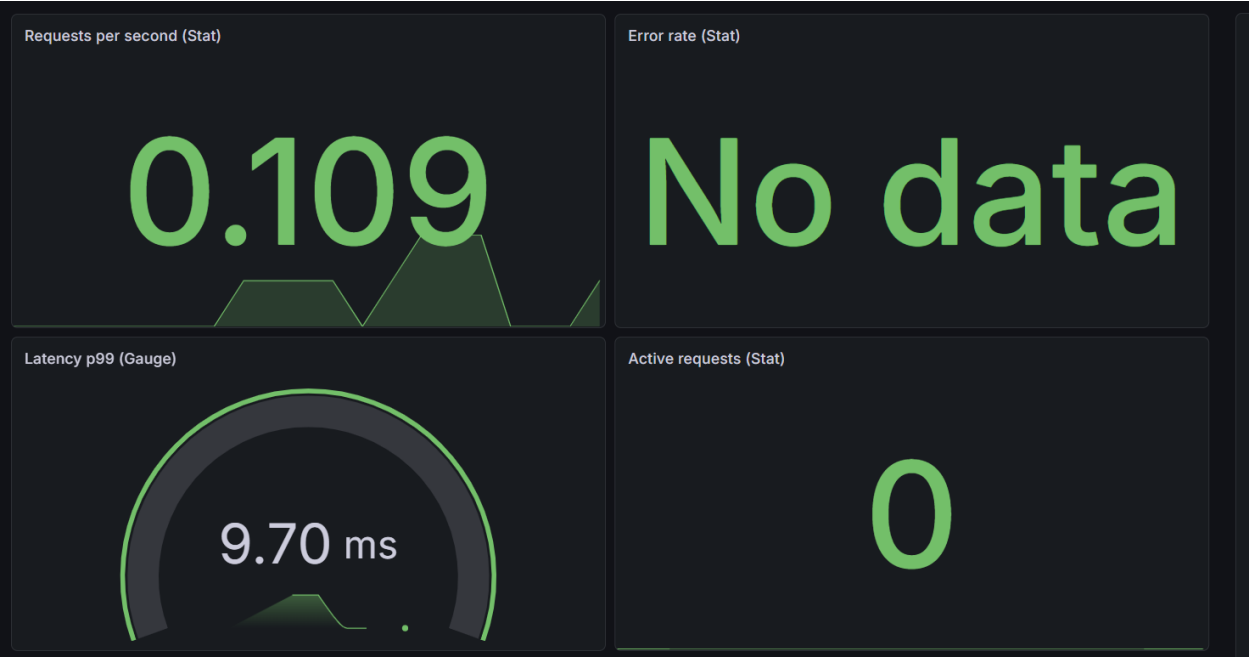
```
docker compose up -d
```

This starts all databases, Redis, NATS, Prometheus, Jaeger, Loki, and Grafana.

PROMETHEUS:



GRAFANA VIZUALIZATION:



```

    },
    {
      "id": "cc872a60-e9a4-4ed4-aa74-79aebb760b5c",
      "title": "Back to the Future",
      "genre": "Adventure",
      "durationMin": 116
    },
    {
      "id": "d61ef9cd-3eed-410d-92eb-bfd25fa1eb0c",
      "title": "Coco",
      "genre": "Animation",
      "durationMin": 105
    },
    {
      "id": "3da58571-855c-4f78-ae3f-e3b4607ab2ab",
      "title": "Dune",
      "genre": "Sci-Fi",
      "durationMin": 155
    },
    {
      "id": "b952f9ed-c4ed-4f63-8eb5-3212c42b5139",
      "title": "Eternal Sunshine of the Spotless Mind",
      "genre": "Romance",
      "durationMin": 108
    },
    {
      "id": "dc1ddf0b-6b8a-41c0-a041-5476c0e54acb",
      "title": "Fight Club",
      "genre": "Drama",
      "durationMin": 139
    },
    {
      "id": "0eeab688-0b47-4897-82c3-5660761b118f",
      "title": "Forrest Gump",
      "genre": "Drama",
      "durationMin": 142
    }
  ],
  "total": 50
}
{}
{

```

```

ons.sobate/ndglAPTOP-UGH130 RINGAS d/cinema-system/notification-service (main)
$ grpcurl -plaintext localhost:50051 movie.MoviesService/ListMovies
grpcurl -plaintext localhost:50052 showtime.ShowtimeService/ListShowtimesByMovie
grpcurl -plaintext -d '{"user_id":"u1","showtime_id":"a1b2c3d4-e5f6-7890-abcd-ef1234567891"}' localhost:50053 booking.BookingService/CreateBooking
{
  "movies": [
    {
      "id": "521a4736-1417-44c7-88f6-fe8669082a9c",
      "title": "A Beautiful Mind",
      "genre": "Biography",
      "durationMin": 135
    },
    {
      "id": "d8fc0377-9b47-4f07-a1e9-12c956217da2",
      "title": "Alien",
      "genre": "Horror",
      "durationMin": 117
    },
    {
      "id": "e3ddd73c-104b-4761-8ca5-89f73f0d00da",
      "title": "American History X",
      "genre": "Drama",
      "durationMin": 119
    },
    {
      "id": "f01340d7-06ce-42d8-8578-e8c131265b97",
      "title": "Avengers: Endgame",
      "genre": "Action",
      "durationMin": 181
    },
    {
      "id": "cc872a60-e9a4-4ed4-aa74-79aebb760b5c",
      "title": "Back to the Future",
      "genre": "Adventure",
      "durationMin": 116
    },
    {
      "id": "d61ef9cd-3eed-410d-92eb-bfd25fa1eb0c",
      "title": "Coco",
      "genre": "Animation",
      "durationMin": 105
    }
  ]
}

```

```
Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-servi
ce (main)
```

```
$ curl -s http://localhost:9101/metrics | grep "grpc_server_handled_tota
]"
```

```
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
```

```
# TYPE grpc server handled total counter
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_s
ervice="movie.MovieService",grpc_type="unary"} 4
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4
```

```
Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-servi
ce (main)
```

```
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total
```

	Dload	Upload	Total	Spent	Left
Speed					

```
0 0 0 0 0 0 0 0 --:--:-- --:--:--100 13773
```

```
0 13773 0 0 1244k 0 # HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
```

```
--# TYPE grpc_server_handled_total counter
```

```
:grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
```

```
-grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionI  
nfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi
```

```
stream"} 4
-:-:-:-:-:-:-:-:-:-:- 1345k
```

```

Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-service (main)
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total
% Total    % Received % Xferd  Average Speed   Time    Time     Time
Current                                  Dload  Upload  Total  Spent    Left
Speed
  0      0  0      0  0      0      0      0  --:--:-- --:--:--100 13772
0 13772  0      0 881915  0  --:--:-- --:--:-- --:--:-- 918133
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
# TYPE grpc_server_handled_total counter
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4

Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-service (main)
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total
% Total    % Received % Xferd  Average Speed   Time    Time     Time
Current                                  Dload  Upload  Total  Spent    Left
Speed
  0      0  0      0  0      0      0      0  --:--:-- --:--:--100 13768
0 13768  0      0 961855  0  --:--:-- --:--:-- --:--:-- 983428
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
# TYPE grpc_server_handled_total counter
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4

```

```
Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-service (main)
```

```
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  
Current
```

```
                                Dload  Upload  Total  Spent  Left  
Speed
```

```
0 0 0 0 0 0 0 0 0 --:--:-- --:--:--100 13768  
0 13768 0 0 367902 0 --:--:-- --:--:-- --:--:-- 372108
```

```
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
```

```
# TYPE grpc_server_handled_total counter
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4
```

```
Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-service (main)
```

```
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  
Current
```

```
                                Dload  Upload  Total  Spent  Left  
Speed
```

```
0 0 0 0 0 0 0 0 0 --:--:-- --:--:--100 13773  
0 13773 0 0 961734 0 --:--:-- --:--:-- --:--:-- 983785
```

```
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
```

```
# TYPE grpc_server_handled_total counter
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4
```

```
Пользователь@LAPTOP-LGQH1I3U MINGW64 /d/cinema-system/notification-service (main)
```

```
$ curl http://localhost:9101/metrics | grep grpc_server_handled_total  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  
Current
```

```
                                Dload  Upload  Total  Spent  Left  
Speed
```

```
0 0 0 0 0 0 0 0 0 --:--:-- --:--:--100 13767  
0 13767 0 0 527976 0 --:--:-- --:--:-- --:--:-- 529500
```

```
# HELP grpc_server_handled_total Total number of RPCs completed on the server, regardless of success or failure.
```

```
# TYPE grpc_server_handled_total counter
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ListMovies",grpc_service="movie.MovieService",grpc_type="unary"} 4
```

```
grpc_server_handled_total{grpc_code="OK",grpc_method="ServerReflectionInfo",grpc_service="grpc.reflection.v1.ServerReflection",grpc_type="bidi_stream"} 4
```